

เอกสารประกอบการสอน • ค่าย สอวน. / การแข่งขัน

การเรียกฟังก์ชันเวียนเกิด

Recursion in C++

ทฤษฎี • กลไกหน่วยความจำ • การวิเคราะห์ความซับซ้อน

Divide & Conquer • Backtracking • เรียกเวียนบนโครงสร้างหลากหลาย

พร้อมบทพิสูจน์ การไล่ trace และโจทย์กว่า 25 แบบ

แผนที่การเรียนรู้ 6 บท

1

รากฐาน & วิธีคิด

นิยาม • อุปนัย • leap of faith • head/tail/tree • mutual

2

กลไก & ความซับซ้อน

stack frame • overflow • แก้ recurrence • Master Theorem

3

ตัวอย่างพื้นฐาน

sum • digits • factorial • power • GCD • บทพิสูจน์

4

เจาะลึก Fibonacci

tree • นับ call • memo/DP • fast doubling • Binet

5

โครงสร้าง & เทคนิค

array/string/grid • D&C • Backtracking • Hanoi

6

ฝึกฝน

โจทย์ 3 ระดับ + เกล็ดลับตั้งสมการ

เป้าหมาย: เข้าใจ “ทำไมมันได้ผล” (พิสูจน์ได้) • มองการทำงานในหัวออก • วิเคราะห์ต้นทุน • เลือกใช้ให้ถูกปัญหา

CHAPTER 1

บทที่ 1

รากฐานและวิธีคิดแบบเวียนเกิด

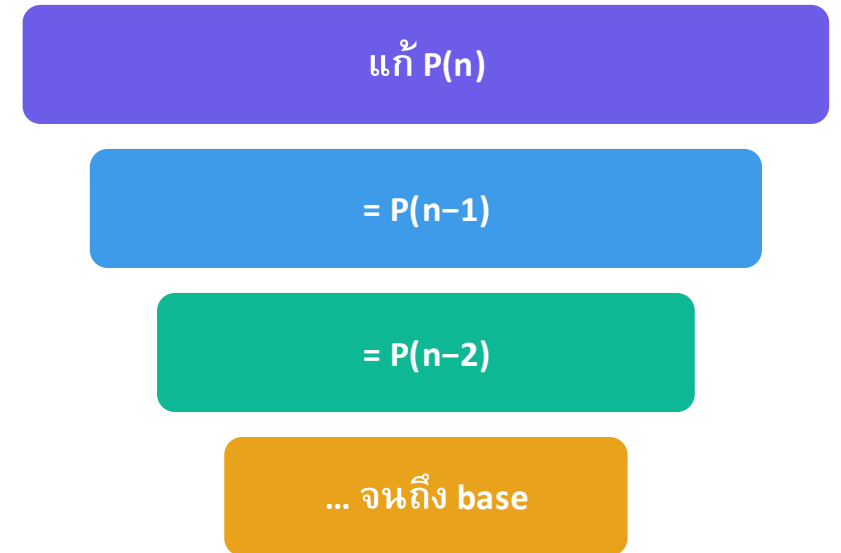
ก่อนเขียนโค้ด ต้อง “คิดแบบเวียนเกิด” ให้เป็นก่อน — เวียนเกิดคือญาติสนิทของการพิสูจน์แบบอุปนัย

Recursion คืออะไร

การเวียนเกิด (Recursion) คือเทคนิคที่ฟังก์ชัน **เรียกใช้ตัวเอง** เพื่อแก้ปัญหา โดยแตกปัญหาใหญ่ให้เป็นปัญหาย่อยที่มีรูปแบบเดียวกันแต่เล็กลง จนเล็กจนตอบได้ทันที

หัวใจ: “นิยามสิ่งหนึ่งด้วยตัวมันเองที่เล็กลง”

- $n! = n \times (n-1)!$ นิยาม factorial ด้วย factorial ที่เล็กลง
- แฟ้มในโฟลเดอร์ = แฟ้มชั้นนี้ + (โฟลเดอร์ย่อยซึ่งก็เป็นโฟลเดอร์อีก)
- โครงสร้างต้นไม้ = โหนด + ต้นไม้ย่อยซ้าย + ต้นไม้ย่อยขวา
- ตึกตาแม่ลูกดก เปิดออกมาเจอตึกตาใบเล็กลงหน้าตาเหมือนเดิม



ปัญหาถูกทำให้ **เล็กลงทุกครั้ง** และวิ่งเข้าหา “กรณีพื้นฐาน” เสมอ

เวียนเกิด = การพิสูจน์แบบอุปนัย (Induction)

ทุกฟังก์ชันเวียนเกิดที่ถูกต้อง คือการพิสูจน์แบบอุปนัยเชิงคณิตศาสตร์ที่ “ทำงานได้จริง” **โครงสร้างตรงกันเป๊ะ:**

กรณีฐาน (Base Case)

ฐานของอุปนัย $P(0)$ จริง

พิสูจน์ว่ากรณีเล็กสุดถูกต้อง

สมมติฐานอุปนัย

สมมติ $P(k)$ จริง

เชื่อว่า $f(k)$ ให้คำตอบถูก (leap of faith)

ขั้นอุปนัย

$P(k) \Rightarrow P(k+1)$

ใช้ $f(k)$ สร้าง $f(k+1)$ ให้ถูกต้อง

ผลลัพธ์: ถ้า base ถูก และขั้นอุปนัยถูก $\rightarrow$ ฟังก์ชันถูกต้องสำหรับ n ทุกค่า “โดยอัตโนมัติ” (ไม่ต้องไล่ทีละขั้น)

หลัก “เชื่อมั่นในตัวเล็กกว่า” (Recursive Leap of Faith)

อย่าพยายามไล่ตามการเรียกซ้ำในหัวทุกชั้น ให้ “สมมติว่า f ของปัญหาที่เล็กกว่าทำงานถูกต้องแล้ว” แล้วถามแค่ที่เราจะต้องตอบเติมอย่างไร

3 คำถามที่ต้องตอบ

- **base case** คืออะไร? กรณีเล็กสุดที่ตอบได้ทันที
- สมมติ f (เล็กกว่า) ถูก มันคืนอะไรกลับมาให้เรา?
- ประกอบอย่างไร? เอาผลย่อยมาต่อเป็นคำตอบของ n

ตัวอย่าง: $\text{sum}(n)$

- base: $\text{sum}(0) = 0$
- สมมติ $\text{sum}(n-1)$ ให้ผลรวม $1..(n-1)$ ถูกต้อง
- แล้ว $\text{sum}(n) = n + \text{sum}(n-1)$ ก็ต้องถูก
- จบ — ไม่ต้องไล่ในหัวว่า $\text{sum}(1), \text{sum}(2), \dots$

ทุกฟังก์ชันเวียนเกิดมี 2 ส่วนเสมอ

1 • กรณีพื้นฐาน (Base Case)

เงื่อนไขที่ **หยุด** การเรียกตัวเอง — ตอบได้ทันที ต้องมีอย่างน้อย 1 เงื่อนไข



```
if (n == 0) return 1;
```

2 • กรณีเวียนเกิด (Recursive Case)

เรียกตัวเองด้วยปัญหาที่ **เล็กลง** แล้วประกอบผลลัพธ์ย่อยกลับเป็นคำตอบเต็ม



```
return n * factorial(n - 1);
```

⚠️ กฎเหล็ก: กรณีเวียนเกิดต้องทำให้อาร์กิวเมนต์ “เข้าใกล้” กรณีพื้นฐานทุกครั้ง มิฉะนั้นจะเรียกไม่รู้จบ → **Stack Overflow**

รูปแบบการเรียกซ้ำ: Head / Tail / Tree

Tail (ท้าย)

เรียกตัวเองเป็นคำสั่งสุดท้าย ไม่มีงานค้างหลังเรียก

```
void f(int n){
    if(n==0) return;
    cout<<n;
    f(n-1); // คำสั่งสุดท้าย
}
```

→ แปลงเป็นลูปได้ตรง ๆ (TCO)

Head (หัว)

เรียกตัวเองก่อน แล้วค่อยทำงานหลังกลับ

```
void f(int n){
    if(n==0) return;
    f(n-1);
    cout<<n; // หลังเรียก
}
```

→ งานเกิดตอน “ขาขึ้น”

Tree (ต้นไม้)

เรียกตัวเองหลายครั้งต่อชั้น

```
int f(int n){
    if(n<2) return n;
    return f(n-1)
        +f(n-2);
}
```

→ แดกกิ่ง → มักแพงมาก

Tail Recursion แปลงเป็นลูปได้

ถ้า “ไม่มีงานค้างหลังการเรียกตัวเอง” คอมไพเลอร์ (เปิด -O2) มัก optimize ให้เป็นลูป ไม่กิน stack เพิ่ม

```
● ● ● แบบ tail (accumulator)

// tail: ส่งผลสะสมผ่านพารามิเตอร์
int fact(int n, int acc=1){
    if(n==0) return acc;
    return fact(n-1, acc*n);
}
```

```
● ● ● แบบลูป

// ลูปเทียบเท่า
int fact(int n){
    int acc=1;
    for(int i=n; i>0; i--) acc*=i;
    return acc;
}
```

ข้อควรรู้ (แข่งขัน): C++ ไม่รับประกัน TCO เสมอ — โจทย์ที่ลึกมากควรเขียนเป็นลูปหรือ explicit stack เพื่อกัน stack overflow

การส่งค่าสะสม (Accumulator Passing)

แทนที่จะ “รวมผลตอนขาขึ้น” เราพก **ตัวสะสม (accumulator)** ติดไปในพารามิเตอร์ ทำให้เป็น tail recursion และมองเหมือนลูป

```
accumulator

// รวมตอนขาลง
int sum(int n, int acc=0) {
    if(n==0) return acc;
    return sum(n-1, acc+n);
}
```

ข้อดี

- งานเสร็จก่อนถึง base — ขาขึ้นไม่ต้องคำนวณ
- แปลงเป็นลูปง่าย เป็น tail call
- ใช้บ่อยใน reverse, การเดินลิสต์, parser

เทียบ 2 สไตล์ sum(3):

ไม่สะสม: $3+(2+(1+0))$ คำนวณตอนขาขึ้น • สะสม: acc ไล่เป็น $0 \rightarrow 3 \rightarrow 5 \rightarrow 6$ คำนวณตอนขาลง

การเรียกไขว้กัน (Mutual / Indirect Recursion)

ฟังก์ชันไม่จำเป็นต้องเรียกตัวเองโดยตรง — **A เรียก B, B เรียก A** ก็เป็นการเวียนเกิดเช่นกัน

```
isEven / isOdd

bool isEven(int n){
    if(n==0) return true;
    return isOdd(n-1);
}
bool isOdd(int n){
    if(n==0) return false;
    return isEven(n-1);
}
```

ที่พบบ่อย

- ตัวแปลภาษา (parser): $\text{expr} \leftrightarrow \text{term} \leftrightarrow \text{factor}$
- ตรวจสอบไวยากรณ์ / grammar แบบ recursive descent
- เกมทฤษฎี: ท่าเดินของผู้เล่น A/B สลับกัน

ยังต้องมี **base case ที่ไปถึงได้** — ที่นี้คือ $n==0$ ในทั้งสองฟังก์ชัน

CHAPTER 2

บทที่ 2

กลไกภายในและการวิเคราะห์ความซับซ้อน

recursion ไม่ใช่เวทมนตร์ — มันวางเฟรมบน call stack และต้นทุนของมันอ่านได้จากสมการเวียนเกิด

Call Stack ทำงานอย่างไร

ทุกครั้งที่เรียกฟังก์ชัน ระบบสร้าง **Stack Frame** เก็บพารามิเตอร์ ตัวแปรท้องถิ่น และตำแหน่งที่ต้องกลับ วางซ้อนกันแบบ **LIFO** (เข้าทีหลัง-ออกก่อน)

- **ขาลง (เรียกซ้ำ):** push เฟรมใหม่ทับขึ้นไปจนถึง base case
- **ขาขึ้น (คืนค่า):** pop เฟรมบนสุด แล้วส่งผลลัพธ์กลับเฟรมล่าง
- **เฟรมบนสุด** คือฟังก์ชันที่กำลังทำงาน ณ ขณะนั้น



ภายใน Stack Frame มีอะไรบ้าง

แต่ละเฟรม (activation record) จองพื้นที่บน stack เก็บข้อมูลที่ทำให้ฟังก์ชัน “กลับมาทำงานต่อได้ถูกจุด”

หน่วยความจำ (stack โตลงล่าง)



Return address

ตำแหน่งคำสั่งถัดไปในผู้เรียก ที่ต้องกระโดดกลับไปเมื่อ return



พารามิเตอร์

ค่าอาร์กิวเมนต์ของการเรียกครั้งนี้ (แต่ละชั้นมีสำเนาของตัวเอง)



ตัวแปรท้องถิ่น

ตัวแปรภายในฟังก์ชัน (เช่น ans) แยกกันทุกเฟรม



Saved registers / frame ptr

ค่าเดิมของรีจิสเตอร์เพื่อคืนสภาพหลังฟังก์ชันจบ

main()

0x7ff...f0

f(3)

0x7ff...e0

f(2)

0x7ff...d0

f(1)

0x7ff...c0

แต่ละเฟรมกินพื้นที่คงที่ • ลึก n → ใช้ $O(n)$

เมื่อ Stack ล้น: Stack Overflow

โค้ดที่ “ล้น” ทำให้เลิกลง



```
int bad(int n) {  
    if (n == 0) return 0;  
    return bad(n); // n ไม่ลด!  
}
```

n ไม่เคยลดลง → ไม่ถึง base case → **push เฟอร์มไม่หยุดจน stack เต็ม**

วิธีป้องกัน

- มี **base case** ที่ไปถึงได้จริง สำหรับทุกอินพุต
- อาร์กิวเมนต์ต้องลดลงเสมอ ($n-1$, $n/2$...)
- ระวังความลึกมาก ลึก $\sim 10^4$ – 10^5 มักล้นบนเครื่องแข่ง
- ถ้าลึกเกินไป แปลงเป็นลูป/ใช้ stack ของตัวเอง
- ระวัง **Infinite loop** จากเงื่อนไขหยุดที่ครอบไม่ครบ

จาก Recursion เป็น Iteration ด้วย Stack ของเราเอง

ทุก recursion แปลงเป็นลูปได้ **โดยจำลอง call stack ด้วยโครงสร้าง stack เอง** — เลี่ยง stack overflow และคุมหน่วยความจำได้

```
// พิมพ์ n..1 แบบไม่ใช่ recursion
void countdown(int n){
    stack<int> st;
    st.push(n);
    while(!st.empty()){
        int x = st.top(); st.pop();
        if(x==0) continue;
        cout << x << ' ';
        st.push(x-1);
    }
}
```

เมื่อไรควรแปลง

- ความลึกอาจเกินขีดจำกัด stack ระบบ
- ต้องการควบคุมลำดับ/หยุดกลางคันเอง
- DFS บนกราฟ/กริดขนาดใหญ่มาก
- หลีกเลี่ยง overhead การเรียกฟังก์ชัน

แลกมาด้วยโค้ดที่อ่านยากขึ้นเล็กน้อย

อ่านต้นทุนจากสมการเวียนเกิด (Recurrence)

เขียนต้นทุนเป็นสมการ: $T(n) = (\text{งานต่อขั้น}) + (\text{ต้นทุนของการเรียกย่อยทั้งหมด})$

ฟังก์ชัน	Recurrence	เวลา	พื้นที่
factorial(n)	$T(n) = T(n-1) + O(1)$	$O(n)$	$O(n)$
fib(n) เวียนตรง ๆ	$T(n) = T(n-1) + T(n-2) + O(1)$	$O(\varphi^n) \approx O(1.618^n)$	$O(n)$
binary search	$T(n) = T(n/2) + O(1)$	$O(\log n)$	$O(\log n)$
merge sort	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	$O(n)$
Hanoi	$T(n) = 2T(n-1) + O(1)$	$O(2^n)$	$O(n)$

จำง่าย: แยกเป็นสองกิ่งโดยไม่จำผล = ระเบิด exponential • แบ่งครึ่ง = log • ทำงานเชิงเส้นต่อขั้น + แบ่งครึ่ง = $n \log n$

การกางสมการ (Substitution / Unrolling)

แทนค่าซ้ำ ๆ จนเห็นรูปแบบ แล้วสรุปเป็นสูตรปิด — ตัวอย่าง $T(n)=T(n-1)+n$, $T(0)=0$

กางทีละขั้น

$$\begin{aligned}T(n) &= T(n-1) + n \\&= T(n-2) + (n-1) + n \\&= T(n-3) + (n-2) + (n-1) + n \\&= \dots \\&= T(0) + 1 + 2 + \dots + n \\&= n(n+1)/2 \Rightarrow O(n^2)\end{aligned}$$

ขั้นตอน

- แทน $T(n-1)$ ด้วยนิยามของมันเอง
- ทำซ้ำ k ครั้ง หา “รูปแบบทั่วไป”
- แทนจนชนกับ base case
- รวมพจน์ $\rightarrow$ สูตรปิด $\rightarrow$ Big-O

วิธีต้นไม้การเรียก (Recursion Tree)

รวม “งานในแต่ละระดับ” ของต้นไม้ ตัวอย่าง $T(n)=2T(n/2)+n$

ระดับ 0

n

งานรวม = n

ระดับ 1

$n/2 + n/2$

งานรวม = n

ระดับ 2

$4 \times n/4$

งานรวม = n

...

...

งานรวม = n

ระดับ $\log n$

$n \times O(1)$

งานรวม = n

เหมาะกับ D&C ที่แต่ละระดับทำงานรวมเท่ากัน (เช่น merge sort)

แต่ละระดับงาน = n

มี $\log_2 n + 1$ ระดับ

รวม = $n \times \log n$

→ $O(n \log n)$

ทฤษฎีบทแม่บท (Master Theorem)

สำหรับ $T(n) = a \cdot T(n/b) + f(n)$ เทียบ $f(n)$ กับ $n^{\log_b a}$:

กรณี 1

$f(n)$ เล็กกว่า $n^{\log_b a}$

เช่น binary search: $a=1, b=2 \rightarrow \Theta(\log n)$

$$T(n) = \Theta(n^{\log_b a})$$

กรณี 2

$f(n)$ เท่ากับ $n^{\log_b a}$

เช่น merge sort: $a=2, b=2, f=n \rightarrow \Theta(n \log n)$

$$T(n) = \Theta(n^{\log_b a} \cdot \log n)$$

กรณี 3

$f(n)$ ใหญ่กว่า $n^{\log_b a}$

งานต่อขั้นครอบงำ

$$T(n) = \Theta(f(n))$$

ลัดการวิเคราะห์ D&C ได้ทันทีโดยไม่ต้องกางต้นไม้

CHAPTER 3

บทที่ 3

ตัวอย่างพื้นฐาน (พร้อมบทพิสูจน์)

เริ่มจากโจทย์คลาสสิก แต่คราวนี้เพิ่ม “ทำไมมันถูก” และการวิเคราะห์ต้นทุน

ตัวอย่างที่ 1

ผลรวม 1..n + พิสูจน์ความถูกต้อง

```
int sum(int n){  
    if(n==0) return 0;  
    return n + sum(n-1);  
}
```

พิสูจน์ด้วยอุปนัย

- ฐาน: $\text{sum}(0)=0$ = ผลรวมของเซตว่าง ✓
- สมมติ: $\text{sum}(k-1)=1+\dots+(k-1)$ ถูก
- ขั้น: $\text{sum}(k)=k+\text{sum}(k-1)=1+\dots+k$ ✓

$\text{sum}(4) = 4 + \text{sum}(3)$

$\text{sum}(3) = 3 + \text{sum}(2)$

$\text{sum}(2) = 2 + \text{sum}(1)$

$\text{sum}(1) = 1 + \text{sum}(0)$

คิดค่า: $0 \rightarrow 1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \Rightarrow \text{sum}(4)=10$

ต้นทุน

$T(n)=T(n-1)+O(1) \rightarrow$ เวลา $O(n)$

ลึก n ชั้น $\rightarrow$ พื้นที่ stack $O(n)$

มีสูตรปิด $n(n+1)/2$ ($O(1)$) — recursion ใช้ฝึกคิด

ชุดโจทย์ “ตัวเลขทีละหลัก”

นับจำนวนหลัก

```
int digits(int n){  
    if(n<10) return 1;  
    return 1+digits(n/10);  
}
```

ผลรวมของหลัก

```
int dsum(int n){  
    if(n==0) return 0;  
    return n%10 + dsum(n/10);  
}
```

กลับเลข (สร้างสตริง)

```
string rev(int n){  
    if(n==0) return "";  
    return to_string(n%10)  
        + rev(n/10);  
}
```

แนวคิดร่วม: “ตัดหลักสุดท้ายด้วย $n\%10$ แล้วเรียกซ้ำกับ $n/10$ ” — เลขจำนวนถูกทำให้เล็กลงเรื่อย ๆ จนเป็น 0

Factorial : $n!$ + พิสูจน์และ overflow

```
int factorial(int n){  
    if(n==0) return 1;  
    return n*factorial(n-1);  
}
```

พิสูจน์ (อุปนัย)

- ฐาน: $0! = 1$ ตามนิยาม ✓
- สมมติ: $\text{factorial}(k-1) = (k-1)!$ ถูก
- ขั้น: $\text{factorial}(k) = k \cdot (k-1)! = k!$ ✓

ทำไม base = 0 ไม่ใช่ 1?

ครอบ $n \geq 0$ ครอบ • ถ้าใช้ $n=1$ จะพังเมื่อ $\text{factorial}(0)$

ระวัง overflow (แข่งขัน)

$$12! = 479,001,600 < 2^{31}-1$$

13! ล้น int → ใช้ long long (ถึง 20!)

ค่ามากกว่านั้น: คำนวณ mod p

factorial(3) ที่ละขั้น (push/pop)

ขาลง — เรียกซ้ำ (push)

factorial(3)

→ $3 \times \text{factorial}(2)$

factorial(2)

→ $2 \times \text{factorial}(1)$

factorial(1)

→ $1 \times \text{factorial}(0)$

factorial(0)

→ return 1 (base)

ขาขึ้น — คืนค่า (pop)

factorial(3)

→ $3 \times 2 = 6$

factorial(2)

→ $2 \times 1 = 2$

factorial(1)

→ $1 \times 1 = 1$

factorial(0)

→ return 1

ผลลัพธ์: **factorial(3)=6** — “คำนวณจริง” เกิดตอนขาขึ้นหลังชน base case

ตัวอย่างที่ 4

ยกกำลัง a^n : สองสไตล์

เวียนเกิดปกติ

```
// เชิงเส้น  $O(n)$ 
long long power(long long a, int n) {
    if (n == 0) return 1;
    return a * power(a, n - 1);
}
```

แบบ tail

```
// tail + accumulator
long long power(long long a, int n,
                long long acc = 1) {
    if (n == 0) return acc;
    return power(a, n - 1, acc * a);
}
```

ไล่ **power(2,4)**: $2 \cdot \text{power}(2,3) = 2 \cdot 8 \rightarrow 2 \cdot \text{power}(2,2) = 2 \cdot 4 \rightarrow \dots \rightarrow a^0 = 1 \Rightarrow 16$

ทั้งคู่เวลา $O(n)$ พื้นที่ $O(n)$ — คำถามชวนคิด: ถ้า n เป็นพันล้าน จะซ้ำและ stack ล้น มีวิธี $O(\log n)$ ไหม? (สไลด์ถัดไป)

ยกกำลังเร็ว (Exponentiation by Squaring)

ไอเดีย: $a^n = (a^{n/2})^2$ เมื่อ n คู่, $= a \cdot a^{n-1}$ เมื่อ n คี่ $\rightarrow$ เรียกแค่ $\sim \log_2 n$ ชั้น

```
long long power(long long a, long long n) {  
    if(n==0) return 1;  
    long long half = power(a, n/2);  
    if(n%2==0) return half*half;  
    return a*half*half;  
}
```

2^{10} ใช้กี่ชั้น?

- $2^{10} = (2^5)^2$
- $2^5 = 2 \cdot (2^2)^2$
- $2^2 = (2^1)^2$
- แค่ ~ 4 ชั้น vs 10 ชั้น $\rightarrow O(\log n)$

ใช้บ่อยมาก: $a^n \bmod p$, ยกกำลังเมทริกซ์ (เร่ง Fibonacci/DP บนสายอักขระ)

GCD (ยุคลิด) + ทำไมมันถูก

```
int gcd(int a,int b){  
    if(b==0) return a;  
    return gcd(b, a%b);  
}
```

ทำไม $\text{gcd}(a,b)=\text{gcd}(b, a\%b)$?

ให้ $a = qb + r$ ($r=a\%b$)

ตัวหารร่วมของ a,b = ตัวหารร่วมของ b,r

เพราะ $r = a - qb$ หารด้วยตัวเดียวกันลงตัว

$\text{gcd}(1785, 546)$

$1785\%546 = 147$

$\text{gcd}(546, 147)$

$546\%147 = 105$

$\text{gcd}(147, 105)$

$147\%105 = 42$

$\text{gcd}(105, 42)$

$105\%42 = 21$

$\text{gcd}(42, 21)$

$42\%21 = 0 \rightarrow$ ตอบ 21

เร็วแค่ไหน? (ทฤษฎีบทลามะ)

จำนวนขั้น = $O(\log(\min(a,b)))$

กรณีแย่สุดเกิดกับเลข Fibonacci ที่ติดกัน

ต่อยอด: extended Euclid $\rightarrow$ หา modular inverse

CHAPTER 4

บทที่ 4

เจาะลึก Fibonacci

ตัวอย่างเดี่ยวที่สอนได้ทั้ง: ความซ้ำซ้อน, memoization, DP, เทคนิค $O(\log n)$ และสูตรปิด

Fibonacci และลำดับ

$$F_n = F_{n-1} + F_{n-2} , F_0=0 , F_1=1$$

```
int fibo(int n){  
    if(n==0||n==1) return n;  
    return fibo(n-1)+fibo(n-2);  
}
```

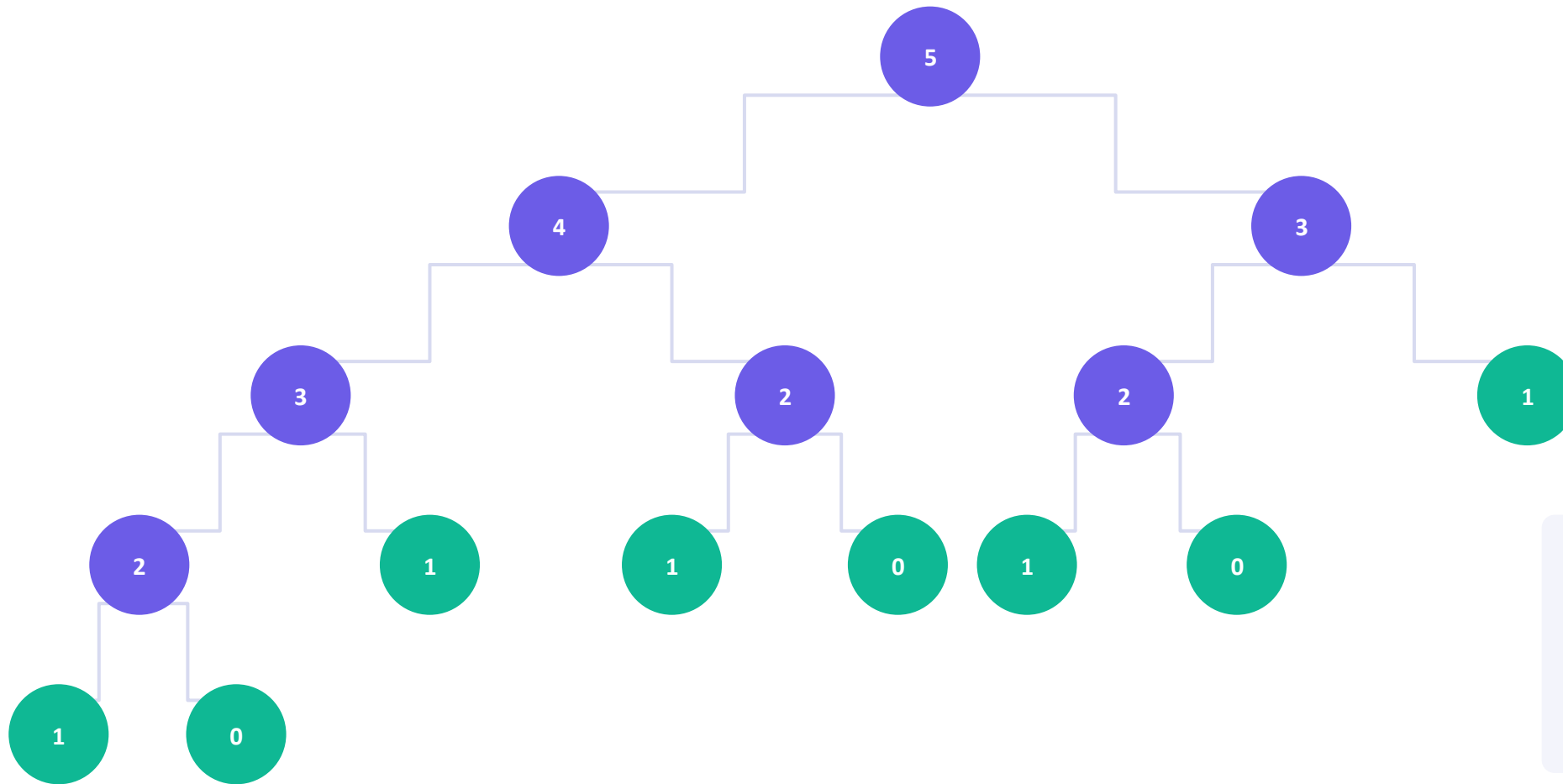
ลำดับฟีโบนัชชี

n	0	1	2	3	4	5	6	7	8
F_n	0	1	1	2	3	5	8	13	21

โค้ดนี้ “สวย” แต่ **ช้ามาก** — เรียกซ้ำงานเดิมนับครั้งไม่ถ้วน (ดูสไลด์ถัดไป)

ทำไมช้า

ต้นไม้การเรียกของ fibo(5)



fibo(3) คำนวณ 2 ครั้ง,
fibo(2) 3 ครั้ง!
→ $O(2^n)$

โหนดเขียว = base case • งานซ้ำทวีคูณเมื่อ n โต

fibo(n) เรียกตัวเองกี่ครั้งกันแน่?

ให้ $C(n)$ = จำนวนครั้งที่ฟังก์ชันถูกเรียกทั้งหมดในการหา $\text{fibo}(n)$

● ● ● สมการของจำนวน call

$$C(0) = C(1) = 1$$

$$C(n) = C(n-1) + C(n-2) + 1$$

n	0	1	2	3	4	5	6
call C(n)	1	1	3	5	9	15	25

สูตรปิด: $C(n) = 2 \cdot F(n+1) - 1$ เติบโตแบบเดียวกับ Fibonacci เอง คือ $\sim \phi^n$

แปลว่า $\text{fibo}(50)$ เรียกตัวเองกว่า 2.5 หมื่นล้านครั้ง — ช้าจนใช้จริงไม่ได้ ต้องปรับปรุง

Memoization และ Bottom-up DP

วิธี A — Memoization (จำคำตอบ)

```
long long memo[100]={0};
long long fib(int n){
    if(n<2) return n;
    if(memo[n]) return memo[n];
    return memo[n]=fib(n-1)+fib(n-2);
}
```

จำผลที่เคยคิด ไม่คิดซ้ำ → **O(n)** (top-down DP)

วิธี B — Bottom-up (ล่างขึ้นบน)

```
long long fib(int n){
    long long a=0,b=1;
    for(int i=0;i<n;i++){
        long long t=a+b; a=b; b=t;
    }
    return a;
}
```

ไล่จากเล็กไปใหญ่ → **O(n)** เวลา, **O(1)** พื้นที่

Fibonacci แบบ $O(\log n)$ และสูตรปิด

Fast Doubling (จาก identity)

```
// คืน (F(n), F(n+1))
pair<ll,ll> fib(ll n){
    if(n==0) return {0,1};
    auto [a,b] = fib(n/2);
    ll c = a*(2*b - a);      // F(2k)
    ll d = a*a + b*b;       // F(2k+1)
    if(n&1) return {d, c+d};
    return {c, d};
}
```

ทางเลือกอื่น

- เมทริกซ์: $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$ ยกกำลังเร็ว $\rightarrow O(\log n)$
- Binet: $F_n = (\phi^n - \psi^n)/\sqrt{5}$
- $\phi = (1+\sqrt{5})/2$ แต่ปัดเศษพียงเมื่อ n ใหญ่

สรุปวิวัฒนาการ Fibonacci: $O(2^n)$ เวียนตรง $\rightarrow O(n)$ DP $\rightarrow O(\log n)$ fast doubling/เมทริกซ์ — โจทย์เดียวสอนได้ทั้งบท

CHAPTER 5

บทที่ 5

เวียนเกิดบนโครงสร้าง & เทคนิคใหญ่

array • string • กริด 2 มิติ • Divide & Conquer • Backtracking • Tower of Hanoi

เวียนเกิดกับ array

```
sum

// ผลรวมสมาชิก a[0..n-1]
int sum(int a[],int n){
    if(n==0) return 0;
    return a[n-1] + sum(a, n-1);
}
```

```
max

// ค่ามากที่สุด
int mx(int a[],int n){
    if(n==1) return a[0];
    return max(a[n-1], mx(a,n-1));
}
```

แพตเทิร์น: “ตัวสุดท้าย” + “ผลของ n-1 ตัวแรก”

แปลงเป็นค่าเฉลี่ย/นับ/ค้นหาได้ทันทีด้วยใจเดียวกัน

เวลา $O(n)$, พื้นที่ $O(n)$ จาก stack — ลูปทำได้ $O(1)$ พื้นที่ แต่ recursion สื่อความคิดชัดกว่า

สร้างทุกลำดับย่อย (Subsequence)

แต่ละตัวอักษร “เลือก” หรือ “ไม่เลือก” → เกิดต้นไม้การตัดสินใจ 2 ทางต่อขั้น

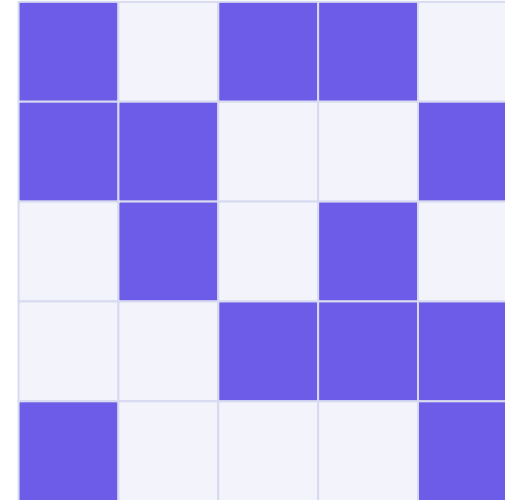
```
void subseq(string s,int i,string cur){
    if(i==s.size()){
        cout << cur << "\n";    // ได้ 1 ชุด
        return;
    }
    subseq(s, i+1, cur+s[i]); // เลือก
    subseq(s, i+1, cur);      // ไม่เลือก
}
```

subseq "AB"

- "" A B AB (4 ชุด)
- n ตัว → 2^n ลำดับย่อย
- ฐาน: i ถึงปลายสตริง → พิมพ์ cur
- หลักเกี่ยวกับการสร้าง “เซตย่อย (subset)”

Flood Fill / ้นบเกาะ (Connected Components)

```
int R,C; char g[105][105];
void flood(int r,int c){
    if(r<0||r>=R||c<0||c>=C) return;
    if(g[r][c]!='#') return;    // ไม่ใช่พื้นที่
    g[r][c]='.';                // ทำเครื่องหมาย
    flood(r+1,c); flood(r-1,c);
    flood(r,c+1); flood(r,c-1);
}
// ้นบเกาะ: วนทุกช่อง เจอ '#' → islands++, flood
```



ช่องสีม่วง = พื้นดินที่เชื่อมกันเป็นเกาะ

แนวคิด DFS: “ทำเครื่องหมายช่องนี้ แล้วลามไป 4 ทิศ” —
เวียนเกิดเดินเต็มพื้นที่ที่เชื่อมกัน

จำนวนเส้นทางในกริด (ขวา/ลงเท่านั้น)

จากมุมซ้ายบนไปมุมขวาล่าง เดินได้เฉพาะ “ขวา” หรือ “ลง” มีกี่เส้นทาง?

```
int paths(int r,int c){
    if(r==0||c==0) return 1;    // ขอบ
    return paths(r-1,c)
        + paths(r,c-1);
}
```

วิเคราะห์

- ฐาน: แถว/คอลัมน์เดียว → มี 1 ทาง
- เวียนตรง ๆ: ซ้ำซ้อน $O(2^{(r+c)})$ — เหมือน Fibonacci
- ปรับ: memoization → $O(r \cdot c)$
- คำตอบปิด: $C(r+c, r)$

บทเรียนซ้ำ: การเรียกที่ “ทับซ้อนกัน” คือสัญญาณว่าให้ทำ memoization / DP

ปัญหาโยเซฟุส (Josephus)

คน n คนยืนเป็นวง นับข้ามทีละ k แล้วคนนั้นออก **ถามว่าใครรอดเป็นคนสุดท้าย?**

```
// ตำแหน่งผู้รอด (นับจาก 0)
int jos(int n,int k){
    if(n==1) return 0;
    return (jos(n-1,k) + k) % n;
}
```

ทำไมได้ผล

- แก้ปัญหา $n-1$ คนก่อน (เชื่อว่าถูก)
- แล้ว “เลื่อน” คำตอบด้วย $+k$ แบบวน (mod n)
- ฐาน: เหลือ 1 คน $\rightarrow$ ตำแหน่ง 0

ตัวอย่างพลังของการตั้งสมการเวียนเกิดจากปัญหาเล็กกว่า — เขียนสั้นมากแต่แก้ปัญหายาก

Divide & Conquer (แบ่งแยกแล้วเอาชนะ)

1

Divide

แบ่งปัญหาเป็นส่วนย่อย (มักครึ่งหนึ่ง)

2

Conquer

แก้แต่ละส่วนด้วยการเรียกตัวเอง

3

Combine

รวมคำตอบย่อยเป็นคำตอบเต็ม

ตัวอย่าง: Binary Search • Merge Sort • Quick Sort • ยกกำลังเร็ว • fast doubling

Binary Search + กับดัก off-by-one

```

int bs(int a[],int lo,int hi,int x){
    if(lo>hi) return -1;           // ไม่พบ
    int mid=lo+(hi-lo)/2;         // กัน overflow
    if(a[mid]==x) return mid;
    if(x<a[mid]) return bs(a,lo,mid-1,x);
    return bs(a,mid+1,hi,x);
}

```

จุดพลาดที่พบบ่อย

- $(lo+hi)/2$ อาจ overflow → ใช้ $lo+(hi-lo)/2$
- ลืม $mid-1$ / $mid+1$ → วนไม่จบ
- เงื่อนไขหยุด $lo>hi$ (ไม่ใช่ $lo==hi$)
- อาเรย์ต้อง “เรียงแล้ว” เท่านั้น

Invariant (ตัวคงสภาพ): ถ้ามี x อยู่ มันอยู่ในช่วง $[lo, hi]$ เสมอ

แต่ละขั้นตัดครึ่งช่วง → $T(n)=T(n/2)+O(1) = O(\log n)$ • 1,000,000 ตัว ≤ 20 ชั้น

Merge Sort

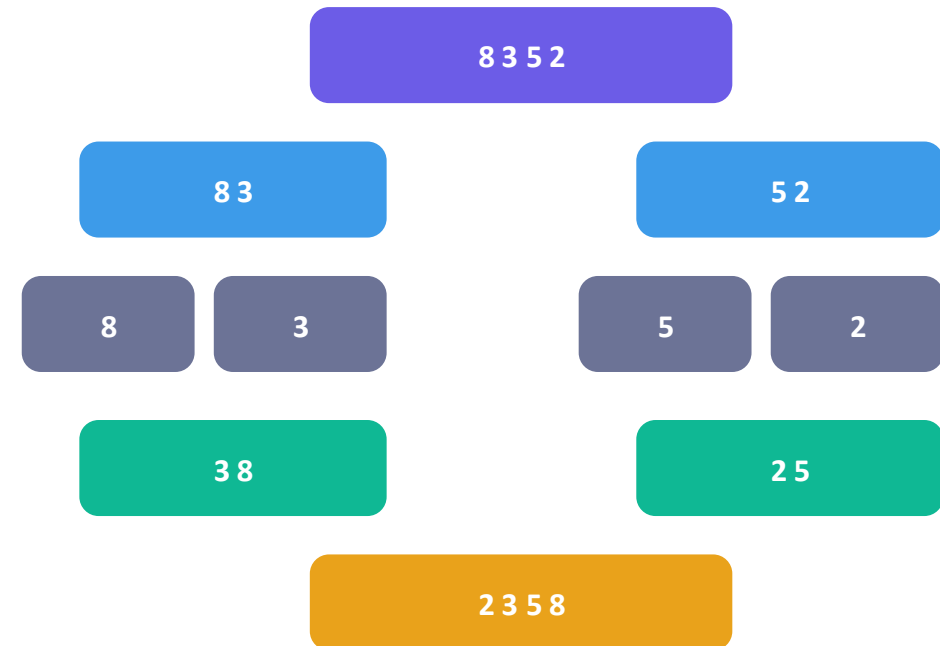
```

void mergeSort(int a[],int l,int r){
    if(l>=r) return;          // 1 ตัว = เรียงแล้ว
    int m=(l+r)/2;
    mergeSort(a,l,m);          // แบ่งซ้าย
    mergeSort(a,m+1,r);        // แบ่งขวา
    merge(a,l,m,r);            // รวมแบบเรียง O(n)
}

```

ต้นทุน: $T(n)=2T(n/2)+O(n) \rightarrow O(n \log n)$ เสถียร รับประกันเร็วเสมอ (ต่างจาก quicksort ที่แย่สุด $O(n^2)$)

[8,3,5,2] → แบ่ง → รวม



Backtracking (ลองผิดลองถูกอย่างมีระบบ)

ลองทางเลือกทีละทาง ถ้าไปต่อไม่ได้ก็ **ถอยกลับ (backtrack)** มาลองทางอื่น

Choose

เลือกทางเลือกหนึ่ง

Explore

เรียกตัวเองเพื่อไปต่อ

Un-choose

ถอนการเลือก ลองทางถัดไป

ใช้แก้: permutation • subset • N-Queens • เชาวงกต • Sudoku • วงเล็บสมดุล

โครงสร้างของ Backtracking

```
void backtrack(State st){
    if (เป็นคำตอบสมบูรณ์) {
        บันทึก/พิมพ์คำตอบ; return;
    }
    for (ทางเลือก c ที่เป็นไปได้) {
        if (!ปลอดภัย(c)) continue; // pruning
        เลือก c; // choose
        backtrack(st'); // explore
        ถอน c; // un-choose
    }
}
```

ทุกโจทย์ *backtracking* ใช้โครงเดียวกันนี้ — เปลี่ยนแค่ 3 จุดข้างบน

3 จุดที่ต้องออกแบบ

- เงื่อนไขคำตอบ เมื่อไรถือว่าครบ
- ทางเลือก แต่ละขั้นมีอะไรให้ลองบ้าง
- การตัดกิ่ง (**pruning**) ตัดทางที่รู้ว่าไปไม่รอด — สำคัญต่อความเร็ว

พิมพ์ทุกการเรียงสับเปลี่ยน (Permutation)

```

void permute(string s, string cur, bool used[]){
    if(cur.size()==s.size()){
        cout<<cur<<"\n"; return;
    }
    for(int i=0;i<s.size();i++){
        if(used[i]) continue;
        used[i]=true;           // choose
        permute(s,cur+s[i],used); // explore
        used[i]=false;          // un-choose
    }
}

```

permute "ABC"

- ABC
- ACB
- BAC
- BCA
- CAB
- CBA

3 ตัว $\rightarrow 3! = 6$ แบบ n ตัว $\rightarrow n!$ แบบ

แพตเทิร์น *choose* $\rightarrow$ *explore* $\rightarrow$ *un-choose* คือหัวใจของ *backtracking* ทุกโจทย์

สร้างเซตย่อยและการจัดหมู่ (Subset / Combination)

```
// เลือกสมาชิก k ตัวจาก 1..n
void comb(int n,int k,int start,
          vector<int>& cur){
    if(cur.size()==k){ print(cur); return; }
    for(int i=start;i<=n;i++){
        cur.push_back(i);      // choose
        comb(n,k,i+1,cur);    // explore
        cur.pop_back();        // un-choose
    }
}
```

comb(4,2)

- 12 13 14 23 24 34 ($C(4,2)=6$)
- ใช้ start กันเลือกซ้ำ/สลับที่
- subset ทั้งหมด: ไม่ตรง k, เลือก/ไม่เลือกแต่ละตัว
- จำนวน subset = 2^n

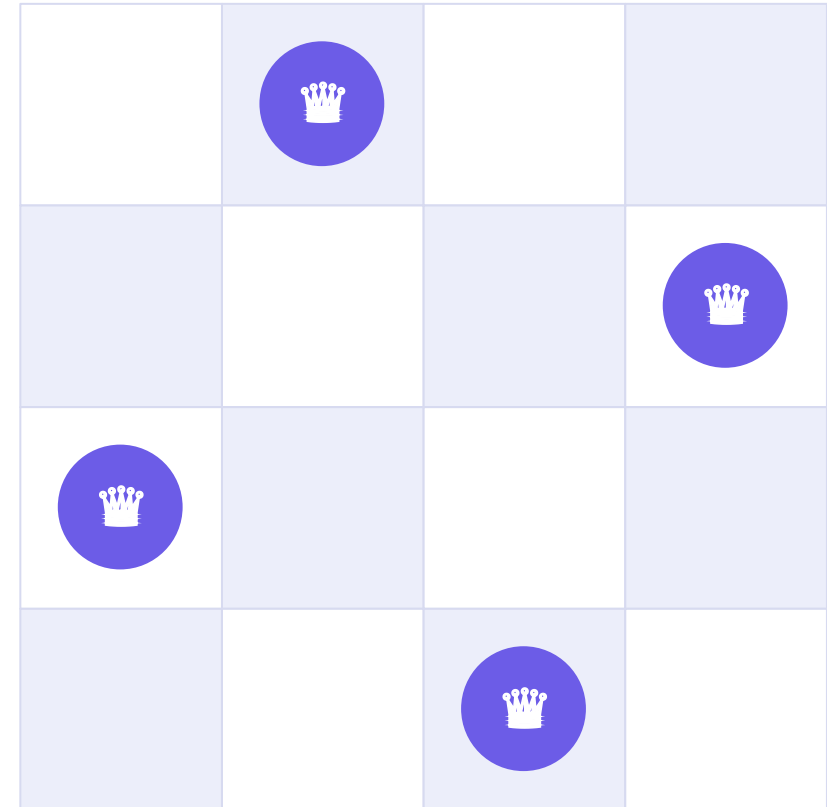
เคล็ดลับ: พารามิเตอร์ start คือ “ตัวตัดกิ่ง” ที่ทำให้ไม่นับชุดเดิมซ้ำ

ปัญหา N-Queens

วางควีน n ตัวบนกระดาน $n \times n$ โดย **ไม่มีคู่ใดกินกันได้** (ไม่ซ้ำแถว/คอลัมน์/ทแยง)

- วางทีละแถว แถวละ 1 ตัว
- ลองทุกคอลัมน์ ถ้า “ปลอดภัย” ก็วางแล้วไปแถวถัดไป
- ถ้าตัน ถอยกลับไปแก้แถวก่อนหน้า (backtrack)
- วางครบ n แถว = พบคำตอบ 1 แบบ

isSafe: เช็ค col, ทแยง (row-col และ row+col) ที่ถูกยึดไว้ — ตัดกิ่งเร็วสำคัญมาก



หนึ่งคำตอบของ 4-Queens

สร้างวงเล็บสมดุลทั้งหมด

ให้วงเล็บ n คู่ พิมพ์ทุกแบบที่สมดุล — ตัวอย่างการ **ตัดกิ่งด้วยเงื่อนไข**

```
void gen(int n, int open, int close, string cur) {
    if (cur.size() == 2 * n) { print(cur); return; }
    if (open < n)                // ยังเปิดได้
        gen(n, open + 1, close, cur + '(');
    if (close < open)            // ปิดได้เมื่อมีคู่เปิด
        gen(n, open, close + 1, cur + ')');
}
```

เงื่อนไข $open < n$ และ $close < open$ คือการ “ตัดกิ่ง” ไม่ให้สร้างสตริงที่เป็นไปไม่ได้

$n = 3$

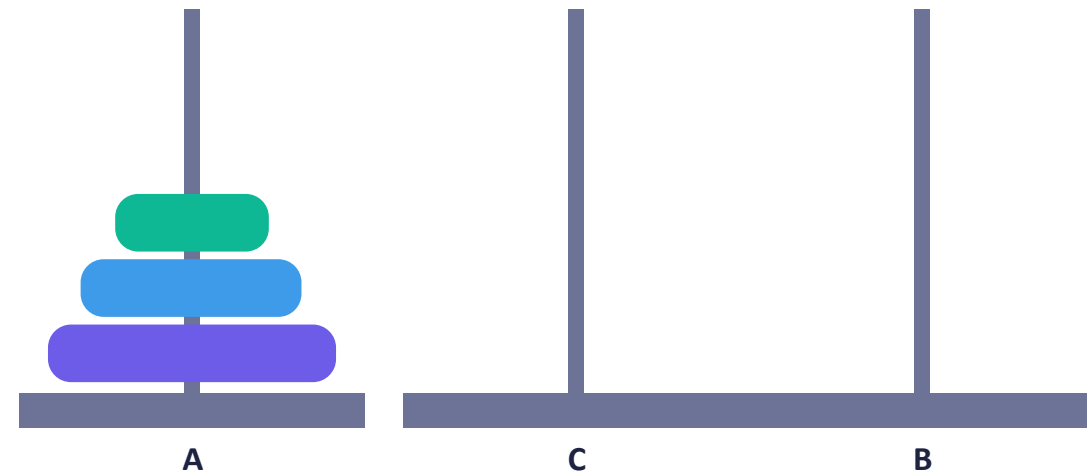
- ((())) (() ()) (())
- () () () () () () (6 แบบ)
- กฎ: ปิดได้ก็ต่อเมื่อ $close < open$
- จำนวนแบบ = เลขคาตาลัน C_n

หอคอยแห่งฮานอย (Tower of Hanoi)

เกมคณิตศาสตร์: หมุด 3 แท่ง (A, B, C) และจาน n ใบขนาดต่างกัน

- เป้าหมาย: ย้ายจานทั้งกองจาก A ไป C
- กติกา 1: ย้ายครั้งละ 1 ใบ (ใบบนสุดเท่านั้น)
- กติกา 2: ห้ามวางจานใหญ่ทับจานเล็ก
- A = ต้นทาง, C = ปลายทาง, B = พักชั่วคราว

สถานะเริ่มต้น



ย้าย n ใบ = 3 ขั้นตอนเวียนเกิด

1

ย้าย $n-1$ ใบบน

จาก $A \rightarrow B$ (ใช้ C เป็นที่พัก)



2

ย้ายจานใหญ่สุด

จาก $A \rightarrow C$ (ที่เดียว)



3

ย้าย $n-1$ ใบ

จาก $B \rightarrow C$ (ใช้ A เป็นที่พัก)

สังเกต: ขั้นที่ 1 และ 3 คือ “ปัญหาเดิมแต่เล็กลง” ($n-1$ ใบ) — หัวใจของการเวียนเกิด

กรณีพื้นฐาน: ถ้ามีจาน 1 ใบ $\rightarrow$ ย้าย $A \rightarrow C$ ได้เลย จบทันที

Tower of Hanoi และจำนวนครั้ง 2^n-1

```
void hanoi(int n,char from,char to,char aux){
    if(n==1){
        cout<<"ย้าย disk 1: "<<from<<" -> "<<to<<"\n";
        return;
    }
    hanoi(n-1, from, aux, to);    // (1)
    cout<<"ย้าย disk "<<n<<": "<<from<<" -> "<<to<<"\n";
    hanoi(n-1, aux, to, from);    // (3)
}
```

พิสูจน์ $M(n)=2^n-1$

$$M(n) = 2 \cdot M(n-1) + 1$$

ฐาน: $M(1)=1 = 2^1-1 \checkmark$

สมมติ $M(k)=2^k-1$

$$M(k+1)=2(2^k-1)+1$$

$$= 2^{k+1}-1 \checkmark$$

$n=3 \rightarrow 7, n=10 \rightarrow 1,023, n=64 \rightarrow \approx 1.8 \times 10^{19}$ ครั้ง (exponential)

ตำนาน: พระ 64 จาน ย้ายวันละ 1 ครั้ง ใช้เวลากว่า 5.8 แส่นล้านปี

CHAPTER 6

บทที่ 6

จากเวียนเกิดสู่ Dynamic Programming

เมื่อการเรียก “ทับซ้อนกัน” — จำผลไว้แล้วปัญหายากกลายเป็นง่าย

สัมประสิทธิ์ทวินาม $C(n,k)$ แบบปาสกาล

$$C(n,k) = C(n-1,k-1) + C(n-1,k) \quad , \quad C(n,0)=C(n,n)=1$$

```
long long C(int n,int k){  
    if(k==0||k==n) return 1;  
    return C(n-1,k-1)+C(n-1,k);  
}
```



การเรียกทับซ้อนกันเยอะ (สามเหลี่ยมปาสกาล) → **memoization** ตาราง $dp[n][k]$ ทำให้เหลือ $O(n \cdot k)$

เวียนตรง ๆ ซ้ำแบบ exponential เช่นเดียวกับ Fibonacci

ทอนเงินได้กี่วิธี (Coin Change)

มีเหรียญชนิด coins[] จ่ายเงิน amount ได้กี่วิธี (ไม่สนลำดับ)?

```
int ways(int amt, int i, int coins[], int n){
    if(amt==0) return 1; // จ่ายพอดี
    if(amt<0 || i==n) return 0; // ไปต่อไม่ได้
    return ways(amt-coins[i], i, coins, n) // ใช้เหรียญ i
        + ways(amt, i+1, coins, n); //ข้ามเหรียญ i
}
```

รูปแบบ “ใช้/ไม่ใช้” นี้พบได้ทั่วไป: knapsack, subset-sum, การเลือกของ

สองทางเลือกต่อขั้น

- ใช้เหรียญ i (ยังใช้ซ้ำได้ $\rightarrow i$ คงเดิม)
- ข้ามเหรียญ i (ไป $i+1$)
- ฐาน: $amt==0 \rightarrow 1$ วิธี
- ทับซ้อน $\rightarrow$ memo $dp[amt][i] = DP$ คลาสสิก

CHAPTER 7

บทที่ 7

สรุป เกล็ดลับ และโจทยฝึก

รวมยอดกับดักที่พบบ่อย วิธีตั้งสมการเวียนเกิด และโจทย์ 3 ระดับ

กับดักที่พบบ่อยของ Recursion



ลืม base case / ครอบไม่ครบ

เรียกไม่รู้จบ → Stack Overflow



อาร์กิวเมนต์ไม่เข้าใกล้ base

เช่น เรียก $f(n)$ แทน $f(n-1)$



คำนวณซ้ำซ้อน

เช่น Fibonacci $O(2^n)$ → memoization



ความลึกมากเกินไป

ลึก $\sim 10^5$ ชนมัก stack ล้น → ลูป



ลำดับคำสั่งก่อน/หลังการเรียก

สลับที่ = ผลลัพธ์เปลี่ยน



ชนิดข้อมูลล้น

factorial/power โตเร็ว → long long / mod

คิดโจทย์เวียนเกิดอย่างไรให้ออก

1

ถามหา “กรณีเล็กสุด”

อะไรคือกรณีที่ตอบได้ทันที? → base case

2

สมมติ “ตัวเล็กกว่าทำได้แล้ว”

ถ้า $f(n-1)$ ให้คำตอบ จะต่อยอดเป็น $f(n)$ อย่างไร?

3

เขียนเป็นสมการเวียนเกิด

เช่น $f(n)=f(n-1)+n$ แล้วแปลงเป็นโค้ด

4

ตรวจว่าเข้าใกล้ base เสมอ

ทุกการเรียกต้องทำให้ปัญหาเล็กลงจริง

5

ไล่มือกับ n เล็ก ๆ

ลองด้วย $n=1,2,3$ เพื่อยืนยันความถูกต้อง

สิ่งที่ต้องจำให้ขึ้นใจ

- Base case + Recursive case ที่เล็กลง = โครงเดียวกับการพิสูจน์อุปนัย
- Call Stack ทำงานแบบ LIFO — เข้าใจ stack frame ช่วยไล่โค้ด
- อ่าน recurrence + Master Theorem เพื่อประเมินเวลา/พื้นที่
- การเรียกทับซ้อน = สัญญาณให้ทำ memoization / DP
- Divide & Conquer: แบ่งครึ่ง $\rightarrow O(\log n)$ หรือ $O(n \log n)$
- Backtracking: choose $\rightarrow$ explore $\rightarrow$ un-choose + ตัดกิ่ง

ก้าวต่อไป $\rightarrow$ ลงมือทำโจทย์ฝึก แล้วไล่ trace ทุกข้อด้วยตัวเอง

โจทย์ Warm-up

W1

พิมพ์ * เป็น n แถว

แถวที่ i มีดาว i ตัว (เวียนเกิด ไม่ใช่ลูปซ้อน)

W2

หาผลคูณ $1 \times 2 \times \dots \times n$

จนกว่าผู้ใช้กด Enter เพื่อหยุด

W3

ค่าเฉลี่ยของ n ตัวใน array

ด้วยฟังก์ชันเวียนเกิด

W4

นับจำนวนหลักของ n

เช่น $4092 \rightarrow 4$ หลัก

W5

ผลรวมของหลักทั้งหมด

เช่น $1234 \rightarrow 10$

W6

พิมพ์ $1..n$ และ $n..1$

ด้วยฟังก์ชันเดียว (สังเกตตำแหน่งพิมพ์)

โจทย์ระดับกลาง

M1

Factorial / Fibonacci

รองรับค่ามากด้วย long long + ทำ memo

M2

เลขยกกำลัง a^n

ทั้งแบบ $O(n)$ และ fast $O(\log n)$

M3

GCD และ LCM

ด้วยยุคลิด + เขียน extended Euclid

M4

แปลงเลขฐานสิบเป็นฐานใด ๆ

ด้วยเวียนเกิด

M5

สร้างทุก subset / permutation

ของชุดข้อมูลที่กำหนด

M6

Binary Search เวียนเกิด

คืนตำแหน่งที่พบ หรือ -1

โจทย์ท้าทาย (แนวแข่งขัน)

C1

Tower of Hanoi

แสดงทุกขั้น + นับจำนวนครั้ง (พิสูจน์ $2^n - 1$)

C2

N-Queens

นับ/แสดงจำนวนวิธีบนกระดาน $n \times n$

C3

น้ำเกาะ / Flood Fill

บนกริดอักขระ (DFS เวียนเกิด)

C4

Coin Change / นับเส้นทางกริด

เวียนเกิด แล้วเร่งด้วย memoization

C5

$a^n \bmod p$ แบบเร็ว

fast exponentiation กับเลขมาก

C6

Merge Sort + นับ inversion

เรียงพร้อมนับคู่สลับ